

Python Programming Fundamentals

Dictionary Basics

Programming Fundamentals Team

SETU

2026-10-13



Outline

1. Creating a Dictionary
2. Accessing Values
3. Safer Access with get()
4. Adding and Updating
5. Useful Dictionary Methods



Outline

1. Creating a Dictionary
2. Accessing Values
3. Safer Access with get()
4. Adding and Updating
5. Useful Dictionary Methods



1. Creating a Dictionary

```
movie = {
    "title": "Jaws",
    "year": 1975,
    "rating": 5
}
```



1. Creating a Dictionary

```
movie = {
    "title": "Jaws",
    "year": 1975,
    "rating": 5
}
```

A dictionary uses:

```
key: value
```

Outline

1. Creating a Dictionary
2. Accessing Values
3. Safer Access with get()
4. Adding and Updating
5. Useful Dictionary Methods



2. Accessing Values

```
print(movie["title"]) # Jaws
print(movie["year"]) # 1975
print(movie["rating"]) # 5
```



2. Accessing Values

```
print(movie["title"]) # Jaws
print(movie["year"]) # 1975
print(movie["rating"]) # 5
```

If the key does not exist, Python raises a `KeyError`.

```
print(movie["director"]) # KeyError
```

Outline

1. Creating a Dictionary
2. Accessing Values
3. Safer Access with get()
4. Adding and Updating
5. Useful Dictionary Methods



3. Safer Access with get()

```
movie = {"title": "Jaws", "year": 1975}

print(movie.get("title")) # Jaws
print(movie.get("title", "Unknown Title")) # Jaws

print(movie.get("director")) # None
print(movie.get("director", "Unknown director"))# Unknown director
```



3. Safer Access with get()

```
movie = {"title": "Jaws", "year": 1975}

print(movie.get("title")) # Jaws
print(movie.get("title", "Unknown Title")) # Jaws

print(movie.get("director")) # None
print(movie.get("director", "Unknown director"))# Unknown director
```

`get()` is useful when a key might not exist because it will return `None` or a default value instead of raising an error.



Outline

1. Creating a Dictionary
2. Accessing Values
3. Safer Access with get()
4. **Adding and Updating**
5. Useful Dictionary Methods



4. Adding and Updating

```
movie = {"title": "Jaws", "year": 1975}

movie["rating"] = 5 # Add a new key-value pair
movie["year"] = 1976 # Update an existing key

print(movie) # {'title': 'Jaws', 'year': 1976, 'rating': 5}
```



4. Adding and Updating

```
movie = {"title": "Jaws", "year": 1975}

movie["rating"] = 5 # Add a new key-value pair
movie["year"] = 1976 # Update an existing key

print(movie) # {'title': 'Jaws', 'year': 1976, 'rating': 5}
```

The same syntax is used to add or update.



Outline

1. Creating a Dictionary

2. Accessing Values

3. Safer Access with get()

4. Adding and Updating

5. Useful Dictionary Methods



5. Useful Dictionary Methods

```
student = {
    "name": "Ava",
    "age": 19,
    "course": "Computing"
}


print(student.keys()) #dict_keys(['name', 'age', 'course', 'mark'])
print(student.values()) #dict_values(['Ava', 19, 'Computing', 85])
print(student.items()) # dict_items([('name', 'Ava'), ('age', 19),
('course', 'Computing'), ('mark', 85)])
```



5. Useful Dictionary Methods

```
student = {
    "name": "Ava",
    "age": 19,
    "course": "Computing"
}


print(student.keys()) #dict_keys(['name', 'age', 'course', 'mark'])
print(student.values()) #dict_values(['Ava', 19, 'Computing', 85])
print(student.items()) # dict_items([('name', 'Ava'), ('age', 19),
('course', 'Computing'), ('mark', 85)])
```



5.1 Looping over key-value pairs:

```
for key, value in student.items():
    print(key, value) # name Ava
                      #age 19
                      #course Computing
                      #mark 82
```

Thanks for Watching - Any questions?

